

Paper Organization. The rest of this paper is organized as follows. Section 2 describes our data curation, code generation, and feature extraction. Section 3 and Section 4 present our analysis of code differences and detection performance. Section 5 examines feature importance and generalization patterns. Section 6 discusses implications, while Section 7, Section 8, and Section 9 address limitations, validity threats, and related work. Section 10 concludes the paper.

2 DATASET

To compare human-written code with code generated by LLMs, we need a data source containing code already authored by human programmers so that we can generate corresponding code using different LLMs for the same tasks. In this section, we explain how we choose our data source, generate corresponding code from four contemporary LLMs, and ensure fair cross-model comparison through careful data curation.

2.1 Data Source

As mentioned before, we break our analyses into two levels: function-level and class-level. In the following, we explain how we prepare the dataset for these two levels of code.

2.1.1 Function-level: In this work, we choose CodeSearchNet [55] as our data source for function-level code. Widely adopted by prior research [14, 45, 47, 78, 84, 91, 107, 119], this corpus enables us to study real-life development through over two million functions mined from open-source GitHub repositories across six programming languages. We specifically utilize the dataset’s (*comment*, *code*) pairs, where *code* denotes the human-written function body and *comment* denotes the corresponding top-level docstring, as illustrated in Listing 2.

Listing 1. Example of a standalone function from pysubs2 [1] with its docstring.

```
def timestamp_to_ms(groups):
    """
    Convert groups from :data:`pysubs2.time.TIMESTAMP` match to milliseconds.
    Example:
    >>> timestamp_to_ms(TIMESTAMP.match("0:00:00.42").groups())
    420
    """
    h, m, s, frac = map(int, groups)
    ms = frac * 10**(3 - len(groups[-1]))
    ms += s * 1000
    ms += m * 60000
    ms += h * 3600000
    return ms
```

CodeSearchNet has over 500,000 Python functions with docstrings. For our study, we focus exclusively on standalone functions. These functions have no dependencies on other functions or classes within the same module. We adopt this constraint because, through preliminary experimentation, we observed that when functions have contextual dependencies (e.g., calling other module-level functions or referencing class attributes), older LLMs (such as GPT-3.5 and Claude 3 Haiku) often fail to generate complete implementations and instead return placeholder code or incomplete snippets. This issue is not unique to our study; Xu *et al.* [111] reported similar challenges when generating code with contextual dependencies. From the available standalone functions, we randomly sample 20,000 functions to limit processing time and cost associated with code generation across multiple LLMs.

2.1.2 Class-level: Functions and classes represent fundamentally different software artifacts with distinct structural characteristics. For example, functions encapsulate algorithmic logic while classes define data structures and behavioural interfaces, which may lead to different generation patterns. Examining both granularities allows us to determine whether detection strategies must be tailored

versions: one human-written version and four LLM-generated versions (one from each model). This results in a total of 72, 425 function-level code snippets and 59, 565 class-level code snippets across all configurations.

Table 1. Data collection and filtering process.

Stage	Function-Level	Class-Level
1. Data Source	CodeSearchNet Dataset	Classes pulled from the SAME OSS Projects
2. Initial Sample	20,000 standalone functions	20,000 standalone classes
3. Code Generation	Generate with 4 LLMs: <i>GPT-3.5</i> , <i>GPT-OSS</i> , <i>Claude 3 Haiku</i> , <i>Claude Haiku 4.5</i>	
4. Validation	Verify valid Python syntax using ast parsing	
5. Intersection Filter	Keep only snippets successfully generated by ALL 4 LLMs	
6. Final Dataset	14,485 functions	11,913 classes

This data curation process ensures that any differences we observe in detection performance or feature importance across models can be attributed to genuine model-specific patterns rather than differences in the difficulty or characteristics of the code snippets themselves.

2.4 Metrics Extraction

Existing works on program comprehension reveal that software metrics can be a valuable source of information for understanding the properties of a piece of software [30, 96, 120]. Building on top of this existing finding, we leverage software metrics from the point of view of distinguishing between human-written and LLM-generated code, because the LLMs might exhibit different code quality characteristics in their generated code [56]. We used *Understand* by *SciTools* [3] to extract software metrics from the functions and classes in our dataset. *Understand* is an industry-standard tool for software analytics with support for all popular programming languages.

We extracted metrics from two main categories: code stylometry metrics (measuring coding style and structure) and code complexity metrics (measuring algorithmic complexity). The specific metrics differ by granularity to capture characteristics relevant at each level. Note that all generated code consists of syntactically valid Python confirmed through ast parsing, requiring no further processing before metric extraction.

2.4.1 *Function-Level Metrics.* For function-level analysis, we used 18 metrics covering:

Code Stylometry (10 metrics): Lines, Blank Lines, Code Lines, Declarative Code Lines, Executable Code Lines, Comment Lines, Statements, Declarative Statements, Executable Statements, Comment to Code Ratio

Code Complexity (8 metrics): Cyclomatic Complexity, Modified Cyclomatic Complexity, Strict Cyclomatic Complexity, Strict Modified Cyclomatic Complexity, Essential Complexity, Maximum Nesting Depth, Paths, Logarithmic Paths

2.4.2 *Class-Level Metrics.* For class-level analysis, we used 39 features covering:

Code Stylometry (18 metrics): Lines, Blank Lines, Code Lines, Declarative Code Lines, Executable Code Lines, Comment Lines, Statements, Declarative Statements, Executable Statements, Comment to Code Ratio, Average Lines, Average Blank Lines, Average Code Lines, Average Comment Lines, Instance Methods, Instance Variables, Methods, All Methods

Code Complexity (21 metrics): Average Cyclomatic Complexity, Average Modified Cyclomatic Complexity, Average Strict Cyclomatic Complexity, Average Strict Modified Cyclomatic Complexity, Average Essential Complexity, Maximum Cyclomatic Complexity, Maximum Modified Cyclomatic